

- *arm-none-eabi-g++*, a wrapper for various phases of C++ development
- *arm-none-eabi-ld*, which links several object files and libraries, generating an executable with the help of linker maps and startup code
- *arm-none-eabi-as*, for assembling code and to generate object files
- *arm-none-eabi-cpp*, for preprocessing source code
- *arm-none-eabi-objcopy*, to convert an executable file from elf format into bin or hex format
- *arm-none-eabi-nm*, to check the symbol table
- *arm-none-eabi-objdump*, which is to disassemble generated object files and section analysis
- *arm-none-eabi-readelf*, for reading the headers based on the magic number present
- *arm-none-eabi-size*, to check the section wise footprint of the generated executable file
- *arm-none-eabi-gdb*, for debugging and tracing with the help of debugging hardware

OpenOCD

This on-chip debugger utility comes with flashing and remote debugging support for various target boards. It works with different JTAG probes connected to the host with onboard or external hardware debuggers. GNU MCU Eclipse OpenOCD is a fork of the original OpenOCD and it is designed to work well with MCU plugins. Prebuilt binaries are available at [gnu-mcu-eclipse.github.io/openocd/releases](https://github.com/gnu-mcu-eclipse/openocd/releases) for various host environments.

Qemu

Qemu, informally known as 'Quick emulator', can execute applications built for various target architectures and can emulate full system images also. Mainline Qemu has limited support for bare metal targets (a few of the TI Stellaris family only) based on the Cortex-M architecture.

GNU MCU Eclipse Qemu is a fork of the original Qemu and comes with support for various popular targets of the STM32 Discovery and Nucleo series. Support for more targets like NXP/Freescale Freedom Kinetis series, NXP LPC series, TI Tiva series, Atmel SAM series and Infineon CPU Card/Board series is in the pipeline for future releases. Please stay tuned to [gnu-mcu-eclipse.github.io/qemu/](https://github.com/gnu-mcu-eclipse/qemu/) for more updates.

Download the latest stable version of Qemu binaries from github.com/gnu-mcu-eclipse/qemu/releases, extract to a suitable location and update the path to the *bin* directory.

IDE with plugins

Download the latest stable version of GNU MCU Eclipse IDE for C/C++ developers from github.com/gnu-mcu-eclipse/org.eclipse.epp.packages/releases. These archives are already bundled with CDT and MCU plugins. The first major release (which is the latest one, as of now) is based on the Neon 3

release of Eclipse IDE and v4.1.1 of MCU plugins.

Alternatively, you can download the regular Eclipse IDE for C/C++ developers using the Eclipse installer or through offline archive options, and update the MCU plugins via Eclipse Market Place. For this, navigate to *Help --> Eclipse Market Place*, search for GNU MCU Eclipse and install the plugins.

These plugins are available at <https://gnu-mcu-eclipse.netlify.com/v4-neon-updates>, which can be specified for manual installation by navigating to *Help --> Install New Software* and adding a new repository with the above URL. This repository supersedes the earlier plugins of v3.x hosted at <http://gnuarmclipse.sourceforge.net/updates>.

The plugins come with support for various targets from the STM32 series, the NXP/Freescale Kinetis series of boards and are capable of the following:

- Generating template projects
- Building the code with the help of a cross toolchain
- Flashing and debugging code using OpenOCD, J-Link or CodeRed modes
- Emulating using Qemu in the absence of hardware

To map the necessary components with the installed plugins, go to *Windows --> Preferences --> MCU* and fill the suitable locations of Toolchain, OpenOCD, Qemu, etc, to enable project wizards to make use of the tools available in these directories.



Note: Actual components like Toolchain, OpenOCD and Qemu need to be installed individually, as mentioned in earlier sections. This plugin set just supports the above components in a friendly mode.

Project templates

The installed plugins (prebundled or manually installed via Market Place) enable project wizards for the STM32 Fxxx series, Freescale Kinetis KLxx series, Analog Devices' ADuCM36X family of boards and a template for the generic Cortex-M series target. These wizards help us to create empty projects or Blinky templates with vendor-specific abstraction APIs and target-specific options under *Project settings*.

Blinky demo

Let's try a Blinky demo for the STM32 F4 Discovery target. This board is equipped with four user LEDs, one user pushbutton, an onboard accelerometer, USB OTG connector, an audio interface and pin out for other interfaces. OpenOCD works well with the onboard ST-Link hardware for debugging purposes. Create a new C project using *File --> New --> C Project*. Next, choose *STM32F4xx C/C++* project as *Type* and give it a suitable name, say *f4-blinky*, as shown in Figure 1.

In the next wizard, choose the desired MCU chip according to the target board and configure other